

Lab # 02

Database and Table Creation with Data Insertion

Objective:

Create a database, a table, and insert sample records into the table.

Introduction:

Once a local server environment (Apache, PHP, and MySQL/MariaDB) has been set up using WampServer, the next step in database development is learning the fundamental SQL Data Definition

Language (DDL) and Data Manipulation Language (DML) commands. These include CREATE DATABASE and CREATE TABLE for defining data structures, and INSERT INTO and SELECT for populating and retrieving data.

This lab uses phpMyAdmin, the browser-based MySQL administration tool bundled with WampServer, to execute SQL statements and observe their results directly. Two tables are created in this lab: a simple Cyber table with just an id and name, and an expanded Cyber25 table that also stores a mobile number and batch, resembling a more realistic student record.

Procedure:

The following steps were carried out to create the database, tables, and insert records. A screenshot was captured after each step (placeholders below to be replaced with actual phpMyAdmin screenshots):

1. Opened phpMyAdmin (<http://localhost/phpmyadmin/>) and ran the following command to create a new database:

```
CREATE DATABASE University;
```



Figure 1: Creating the University database

2. Selected the University database and created the Cyber table with an id column and a name column:

```
CREATE TABLE Cyber ( id  
INT,  
name VARCHAR(100)  
);
```

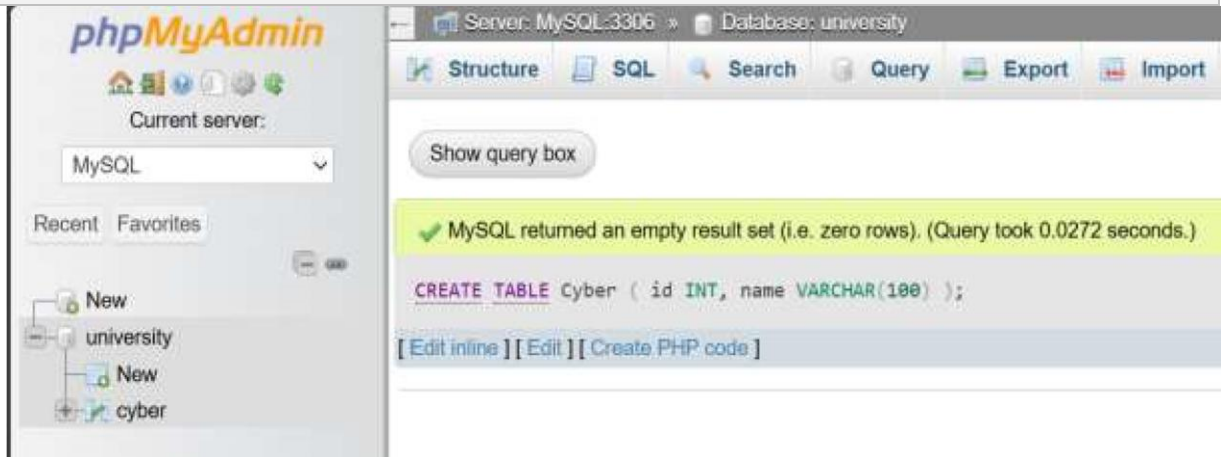


Figure 2: Creating the Cyber table

3. Inserted five sample records into the Cyber table:

```
INSERT INTO Cyber (id, name) VALUES  
(1, 'Ali'),  
(2, 'Ahmed'),  
(3, 'Usman'),  
(4, 'Hassan'),  
(5, 'Bilal');
```



Figure 3: Inserting records into the Cyber table

4. Ran a SELECT query to verify that all records were inserted correctly:

```
SELECT * FROM Cyber;
```

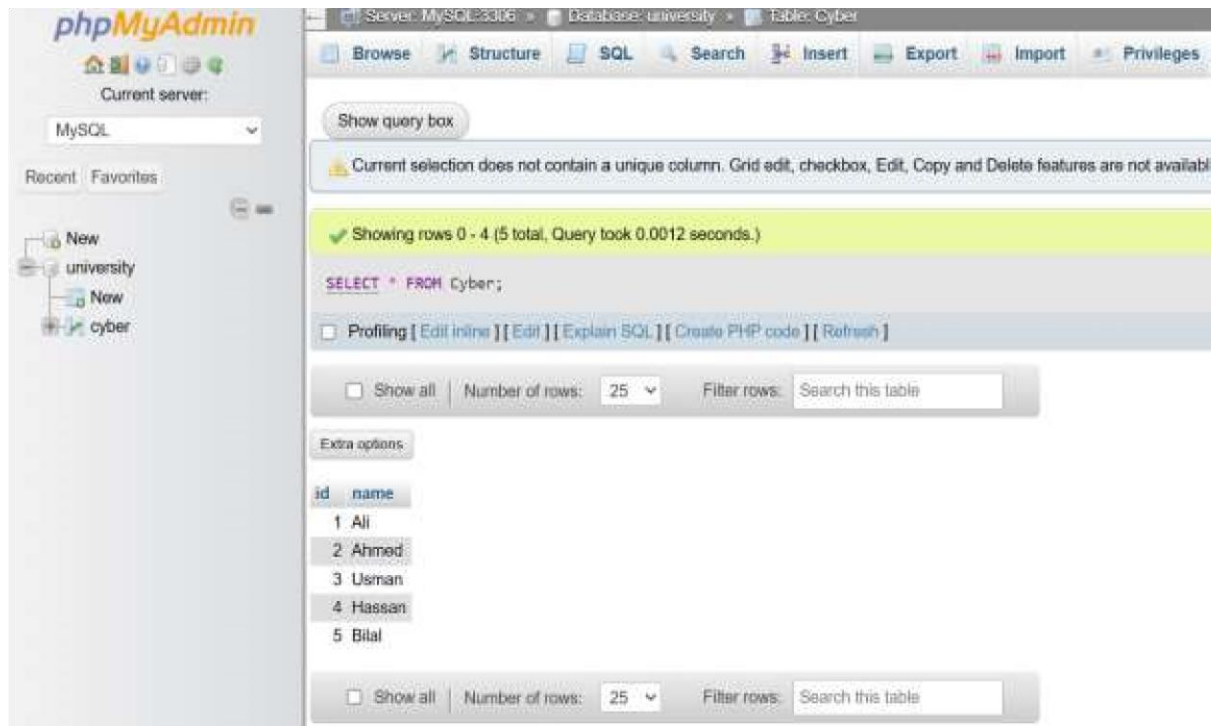


Figure 4: Viewing records in the Cyber table

5. Created a second, more detailed table named Cyber25 within the University database, adding mobile_no and batch columns:

```
CREATE TABLE Cyber25 ( id INT,  
    name VARCHAR(100), mobile_no  
    VARCHAR(15),  
    batch VARCHAR(10)  
);
```

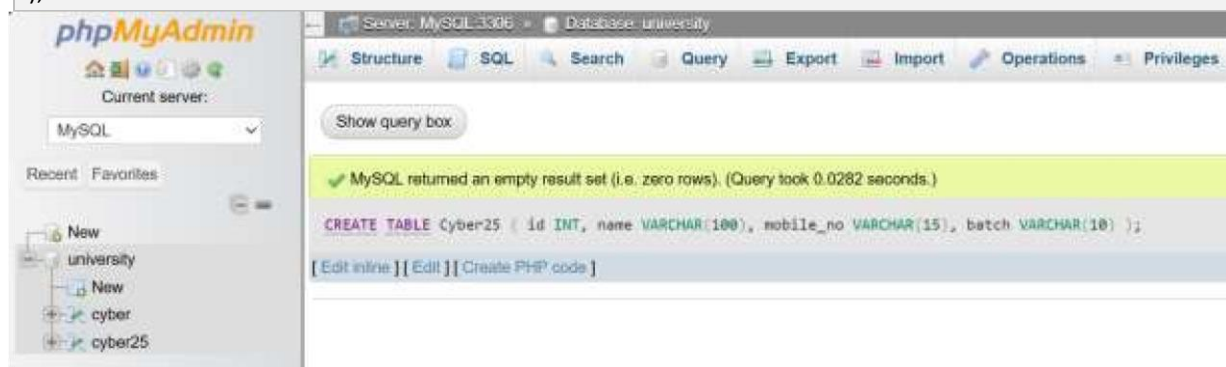


Figure 5: Creating the Cyber25 table

6. Inserted five student records into Cyber25, each including a mobile number and the batch '24F':

```
INSERT INTO Cyber25 (id, name, mobile_no, batch) VALUES  
(1, 'Ali', '03001234567', '24F'),  
(2, 'Ahmed', '03111234567', '24F'),  
(3, 'Usman', '03221234567', '24F'),  
(4, 'Hassan', '03331234567', '24F'),  
(5, 'Bilal', '03441234567', '24F');
```

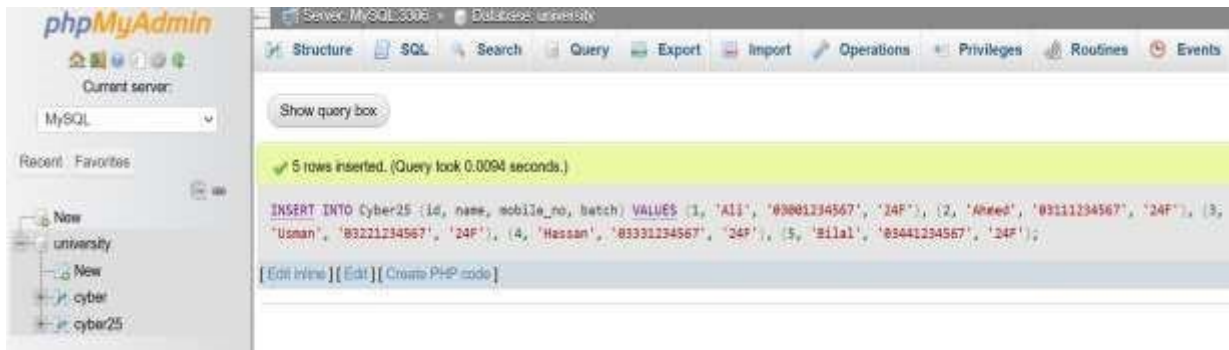


Figure 6: Inserting records into the Cyber25 table

7. Ran a SELECT query on Cyber25 to confirm that all fields, including mobile_no and batch, were stored correctly:

```
SELECT * FROM Cyber25;
```

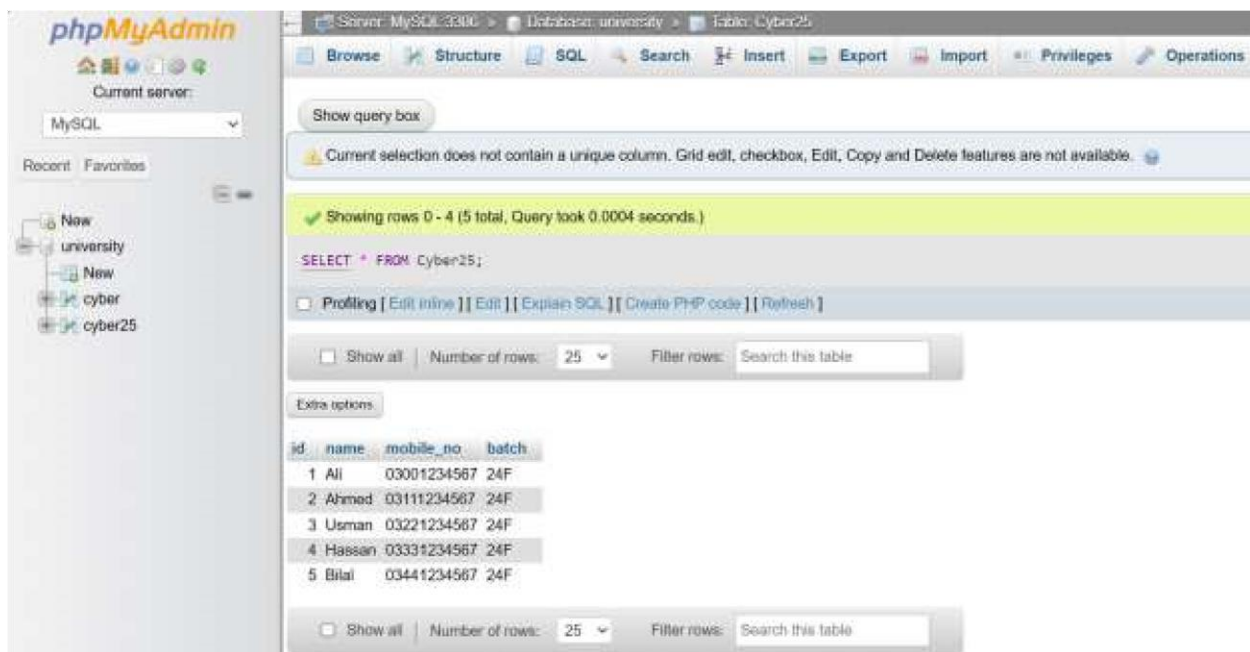


Figure 7: Viewing records in the Cyber25 table

Results:

The University database was created successfully, and both the Cyber and Cyber25 tables were added to it without errors. The Cyber table stored five basic records (id and name), and the `SELECT * FROM Cyber;` query returned all five rows in the order they were inserted.

The Cyber25 table successfully stored the additional mobile_no and batch fields alongside id and name. The `SELECT * FROM Cyber25;` query confirmed that all five records were inserted with the correct phone numbers and the batch value '24F' for every student, demonstrating that VARCHAR columns can store both numeric-looking strings (like phone numbers) and short categorical labels (like batch codes) correctly.

Conclusion:

This lab demonstrated the core SQL commands needed to build and populate a relational database:

`CREATE DATABASE`, `CREATE TABLE`, `INSERT INTO`, and `SELECT`. By first creating a simple twocolumn table and then extending the exercise with a more detailed table containing four columns, this lab illustrated how table structures can be scaled to capture more real-world information, such as contact details and batch/cohort membership. These skills form the foundation for more advanced database operations in future labs, including primary keys, constraints, joins, and relationships between multiple tables.